

Linux 入门（教科书版）

主课本 · 建议按顺序学习 · 与《Linux 指令练习册（基础篇 / 进阶篇）》配套使用

本书是什么：一本从零开始的 Linux 教科书。每个命令都先讲“是什么、为什么存在”，再给**输入 → 输出 → 解读**三段式示范，最后布置作业。

配套关系：先学完本书一章，再翻开练习册做对应阶段的题。**顺序不能反**——练习册是检验，本书是学习。

目录

使用说明 配色约定 · 环境要求 · 安全红线

学习路线图 章 → 练习册映射表

第 0 章 开篇：终端与命令（提示符拆解、三件套、--help 自救）

第 1 章 目录树与路径（一切皆文件、绝对/相对路径）

第 2 章 文件与目录操作（touch/mkdir/cp/mv/rm）

第 3 章 查看与搜索（cat/less/head/tail、grep/find、diff）

第 4 章 管道与重定向（| > >>、sort/uniq、awk 入门）

第 5 章 用户与权限（多用户模型、rwx、chmod 数字算法）

第 6 章 进程管理（ps aux、信号、优雅 vs 强杀、fuser）

第 7 章 磁盘与文件系统（df/du、挂载、fallocate/dd）

第 8 章 网络命令（ip/ping/ss/nc/curl、四层排查法）

第 9 章 apt 包管理（update/install/search、锁报错）

第 10 章 systemctl 与服务（status/start/enable、journalctl）

第 11 章 环境变量（PATH、export、alias、source）

第 12 章 归档与压缩（tar czvf/xzvf、gzip、zip）

第 13 章 Shell 脚本基础（shebang、变量、if/for、crontab）

附录 A 章末作业检查点与自测答案（84 题）

附录 B 术语表（35 条）

附录 C 命令速查总表（按章分组）

附录 D 章 ↔ 练习册对应表（新命令对齐）

打印提示：A4、实际大小（100%）、**关闭浏览器“页眉和页脚”**——本套教材的 PDF 版已自带页码，重复勾选会产生双页码；需要保留彩色底色时，在打印选项里开启“背景图形”。

使用说明

配色约定（全书一致，共 6 色）：

颜色	含义
绿色粗体	命令本身（要敲进去的）
灰色	命令的输出（屏幕上会显示的）
深紫	参数、文件名（命令的宾语，不是输出）
蓝色	解读、分析（帮你理解“为什么”）
红色	危险警告（必须看）
橙色	易错点、坑

实验闭环：涉及删除、重命名、权限、进程、服务、挂载或日志时，先记录原状态；操作中逐步核对对象；操作后对照输出、数量、状态和退出码验证；结束后恢复配置、停止练习进程、卸载实验挂载并再次检查现场。异常时保留原始报错。

环境要求：以电脑 Ubuntu 为主；标注 🌐 的须电脑环境（Ubuntu）完成，平板 Termux 环境下不确定；未标注的平板 Termux 一般可做。

📖 学习日志约定：每天把敲过的命令、报错与心得记进学习日志——这是检查点的依据。

安全红线：实验目录只用 ~/lab 和 /tmp；/tmp 是临时目录，但何时清理由发行版、挂载方式和 systemd-tmpfiles 等系统策略决定，重启后不保证全部清空；rm -rf 永远先 ls 看清楚；涉及 sudo 的命令只在练习机上执行。

学习路线图（章 → 练习册）

🕒 全书 14 章约 22~30 小时（每章约 1.5~2h）。

本书章节	学完去做
第 0~2 章（终端、目录树、文件操作）	基础篇 阶段 1
第 3~4 章（查看搜索、管道重定向）	基础篇 阶段 2
第 5 章（用户与权限）	基础篇 阶段 3
第 6~7、9、10 章（进程、磁盘、服务、包管理）	基础篇 阶段 4
第 3~4、12 章（查找、管道、归档）	基础篇 阶段 5
第 3、8、10 章（grep、网络、服务）	进阶篇 场景 1
第 7、12 章（磁盘、归档）	进阶篇 场景 2
第 8 章（网络）	进阶篇 场景 3
第 6、10 章（进程、服务）	进阶篇 场景 4
第 13 章（Shell 脚本）	进阶篇 场景 5

第 0 章 · 开篇：终端与命令

本章目标（学完能回答）：

① Linux 是什么、为什么运维要学它；② 能打开终端并看懂提示符；③ 会使用"命令 + 选项 + 参数"三件套；④ 会用 `--help` 自救。

本章新命令：`pwd ls echo man clear history exit whoami`（选项 `--help` 见 0.4 节）

0.1 Linux 是什么

Linux 是一个**操作系统**——和你电脑里的 Windows 是同类，负责管理内存、硬盘、CPU 这些硬件，让程序能在上面跑。

Linux 在服务器和云计算环境中使用广泛。它具有**开源、稳定、灵活、便于自动化**等特点，所以 Linux 是运维工作中的重要基础。

你听到的 Ubuntu、CentOS、Debian 叫**发行版**——同一个 Linux 内核，各家包装不同。本书用 **Ubuntu**（最流行、资料最多）。

0.2 打开终端

先搞清楚三个词的关系：**内核 (kernel)** 是 Linux 最核心的程序，管着 CPU、内存、硬盘这些硬件，但你不直接指挥它；**Shell (壳)** 是包在内核外面的"命令翻译官"——你敲的命令先交给它，它翻译成内核能执行的动作；**终端 (Terminal)** 是你看到的那扇"窗户"，把你和 Shell 连起来。以后说"打开终端敲命令"，其实就是"隔着窗户对 Shell 说话"。

终端 (Terminal) 是你和 Linux 对话的窗口。Ubuntu 里按 `Ctrl + Alt + T` 打开。你看到的这一行就是**提示符**：

```
student@ubuntu:~$
```

提示符逐段拆解

```
student@ubuntu:~$
```

student=当前用户名；**@**=在；**ubuntu**=这台机器的主机名；**:**=分隔符；**~**=当前在家目录；**\$**=普通用户（如果是 **#** 说明是管理员 **root**）。看到这串东西出现，说明终端准备好了，在等你敲命令。

0.3 命令三件套

Linux 命令的通用结构，是全书最重要的规律：

↑ **命令 = 动词 + 选项 + 参数**：命令说"做什么"，选项说"怎么做"（改变行为），参数说"对谁做"（目标对象）。

```
$ ls -l /home
```

`ls`(动词：列出) `-l`(选项：以详细格式) `/home`(参数：列这个目录)

选项的规律：短选项用单横线 `-l`，长选项用双横线 `--all`。多个短选项可合并：`ls -la = ls -l -a`。

0.4 自救三招

任何命令卡住了，按顺序试这三招：

招	怎么用
命令 <code>--help</code>	最常用。每个命令都自带简短说明，如 <code>ls --help</code>
<code>man</code> 命令	完整说明书 (manual)，按 <code>q</code> 退出， <code>/</code> 关键词 搜索
外部扩展：搜索引擎	断网时除外；联网时可把报错原文复制去搜，前面加 "ubuntu"。主线靠上面两行 + 第 8、9 册的报错手册自查

0.5 键盘生存技巧（第一天就背）

按键	作用
Tab	自动补全：敲一半按 Tab，能补命令、补文件名
↑ / ↓	翻历史命令（想不起昨天敲过什么就按 ↑）
Ctrl + C	中断当前程序（卡住了按这个）
Ctrl + L	清屏（同 clear ）
Ctrl + A / E	光标跳到行首 / 行尾
Ctrl + R	搜索历史命令
Ctrl + D	退出当前终端（同 exit ）

0.6 示范（三段式）

示范 1：我在哪、我是谁

```
$ pwd
```

/home/<你的真实用户名>（示例输出：实际环境不同，以你机器为准）

pwd = Print Working Directory（打印工作目录）。任何时候迷路了，先敲它看自己在哪。

```
$ whoami
```

student（示例输出，实际以本机为准）

whoami = 我是谁。和提示符里 @ 前面的名字一致。

示范 2：让终端说话

```
$ echo "hello linux"
```

```
hello linux
```

echo = 回声，把后面的话原样打印出来。以后写脚本、看变量全靠它。

示范 3：自救实战

```
$ ls --help
```

```
Usage: ls [OPTION]... [FILE]...
List information about the FILEs...
-l use a long listing format
-a do not ignore entries starting with .
```

读帮助的两条规则：[] 括起来的 = **可选项**（不写也行）；... = **可重复多个**。这行读作“用法：ls、可选选项、可重复的文件名”。

看不懂英文没关系——你现在只需要认出 **-l** 和 **-a** 两个选项的名字。配合《英语单词手册》慢慢积累。

0.7 易错点

坑 1：命令拼错会得到 **command not found**。别慌，不是电脑坏了，是“系统不认识你敲的这个词”——先检查拼写，再想“这个软件装了吗”（第 9 章）、“命令在不在 PATH 里”（第 11 章）。

坑 2：Linux 严格区分大小写。**LS** ≠ **ls**，前者报错。

坑 3：提示符被程序“占住”了？看最后一行有没有命令在跑，按 Ctrl + C 或 q 退出来。

章末作业（答案见附录 A）

1. 打开终端，用 `pwd` 和 `whoami` 各敲一遍，把输出抄进学习日志。
2. 用 `ls --help` 找出 3 个选项，并说出各自作用。
3. 敲 `history`，看你今天敲过的所有命令；用 `↑` 键调出上一条命令再执行一次。
4. 故意敲一个拼错的命令（如 `pws`），把报错原文记下来。

章末自测（答案见附录 A）

1. 提示符 `student@ubuntu:~$` 中，`$` 表示什么？如果换成 `#` 说明什么？
2. 判断：`ls -l /home` 中，`ls` 是选项，`-l` 是命令，`/home` 是参数。（对/错）

 对应练习册：学完本章先做基础篇「阶段 1」里的 1-1、1-2。

第 1 章 • 目录树与路径

本章目标：① 理解"一切皆文件"和目录树；② 说出三大重要目录的用途；③ 会区分绝对路径与相对路径；④ 会用 `cd` 在树里移动。

本章新命令：`cd` (`pwd`、`ls` 在第 0 章已见，本章复习)

1.1 目录树：一棵倒过来的树

Linux 把硬盘上所有东西组织成**一棵树**，树根是一个斜杠 `/`（读作"根目录"）。文件、文件夹（Linux 里叫"目录"）、甚至鼠标硬盘这些设备，全部挂在这棵树上——这就是著名的**"一切皆文件"**。

```
/ ← 树根，一切从这里开始
├─ home/      家目录（每个用户的地盘，你的东西都在这）
│   └─ <你的用户名>/  你的家（提示符里的 ~ 就是指这里）
├─ etc/       配置文件（改设置来这里）
├─ var/log/    日志（排查问题来这里）
├─ tmp/       临时文件；是否在重启后清理由系统配置决定，不能保证全部清空
├─ usr/       已安装的软件
└─ bin/       命令的住处
```

↑ 记住三个排障位置：`/etc` 找配置、`/var/log` 找日志、`/home` 找你的东西。

1.2 两种路径写法

	写法	说明
绝对路径	<code>/home/<你的真实用户名>/notes.md</code>	从树根 <code>/</code> 出发写全称，走到哪都不会认错
相对路径	<code>notes.md</code> 或 <code>../notes.md</code>	从当前位置出发。到达目标的方式取决于"你现在站在哪"
特殊符号	<code>~</code> （我的家） <code>.</code> （当前目录） <code>..</code> （上一级）	三个最常用的"快捷路径"

1.3 示范（三段式）

示范 1：列出家里有什么

```
$ ls -l ~
```

```
total 12
drwxr-xr-x 2 <用户名> <用户名> 4096 Aug 15 08:00 notes
-rw-r--r-- 1 <用户名> <用户名> 123 Aug 15 09:00 hello.py （示例输出，实际以本机为准）
```

每一行是一个条目。**第 1 列**第一个字符：**d**=目录，**-**=普通文件（后面的 `rwX` 权限位第 5 章细讲）。**属主 属组**。**4096/123**=大小（字节）。最后是**修改时间和名字**。

示范 2：在树里移动

```
$ cd /etc # 去 /etc（绝对路径）
$ pwd
/etc
$ cd .. # 回上一级（相对路径）
$ cd ~ # 回家（等价 cd /home/<你的真实用户名>，以 whoami 为准）
$ cd - # 回到刚才的目录（来回切换神器）
```

`cd` = change directory（切换目录）。记住四个快捷：`cd ~` 回家、`cd ..` 上一级、`cd -` 回上一个、`cd`（裸敲）也是回家。

示范 3：看到隐藏文件

```
$ ls -a ~
```

```
.  ..  .bashrc  .profile  notes  hello.py
```

以 `.` 开头的名字是**隐藏文件**（如 `.bashrc` 配置文件，第 11 章会用到）。`ls` 默认不显示它们，加 `-a` 才显示。`.` 和 `..` 就是“当前目录”和“上一级”本身。

1.4 易错点

坑 1： `cd` 后面必须接目录。敲 `cd hello.py` 会报 **Not a directory**。

坑 2： Linux 大小写敏感：Notes 和 notes 是两个不同的东西。

坑 3： 文件名里有空格，要用引号包住：`cd "my notes"`。新手建议文件名永远别用空格，用下划线。

章末作业

1. 用 `cd` 走到 `/etc` 看一眼，再 `cd ~` 回家——两步各用一次 `pwd`，把输出记下来。
2. 从 `/tmp` 出发，分别用绝对路径和相对路径进入 `~/lab/ch1`。
3. 用 `ls -a ~` 找出 3 个隐藏文件，把名字记下来。
4. 用 `cd /` 到树根，用 `ls` 看根下有哪些目录，找找 `/home`、`/etc`、`/var` 在哪。

章末自测

1. 绝对路径和相对路径的核心区别是什么？
2. 判断：`cd ..` 永远回到“上一个待过的目录”。（对/错）

 对应练习册：基础篇 1-1、1-2、1-3。

第 2 章 · 文件与目录操作

本章目标：① 会建文件和目录；② 会复制、移动、改名；③ 理解删除的不可逆；④ 会用 `mkdir -p` 连级创建。

本章新命令： `touch mkdir cp mv rm rmdir`

2.1 创建：touch 和 mkdir

`touch` 造**文件**；`mkdir` 造**目录**（make directory）。`touch` 本义是“碰一下”（更新文件时间戳），文件不存在时顺手就造一个——这个双关用法要记住。

`-p` 是 `mkdir` 最常用选项：**parents**（父级），允许连级创建不存在的中间目录。

示范 1：连级建目录

```
$ mkdir -p ~/lab/ch2/a/b/c
$ ls ~/lab/ch2
a
$ touch ~/lab/ch2/note1.txt ~/lab/ch2/note2.txt
$ ls ~/lab/ch2
a  note1.txt  note2.txt
```

不带 `-p` 时，如果中间的 `a`、`b` 不存在，`mkdir` 会报错；带 `-p` 则一次把整条链建好。`touch` 可以一次造多个文件（空格隔开）。

2.2 复制与移动：cp 和 mv

命令	作用	关键点
<code>cp</code> 源 目标	复制（copy）	复制 目录 必须加 <code>-r</code> （递归 = 连所有子目录一层层一起复制）
<code>mv</code> 源 目标	移动 / 改名（move）	目标路径和源在同一目录 = 改名；跨目录 = 搬家

示范 2：复制目录

```
$ mkdir -p ~/lab/ch2/sub # 先自造一个目录（空目录也能跑完整示范）
$ cp ~/lab/ch2 ~/lab/ch2-copy
cp: -r not specified; omitting directory '/home/你的真实用户名/lab/ch2'（shell 已把 ~ 展开成绝对路径）
```

报错了。`cp` 默认只复制文件；复制目录必须带 `-r`。这就是“先看报错再改正”的标准学习流程。（报错里显示的是你机器上的真实绝对路径，不是波浪号。）

```
$ cp -r ~/lab/ch2 ~/lab/ch2-copy
$ ls ~/lab
ch2  ch2-copy
```

`-r` = recursive（递归），意思是“把这个目录连同里面所有东西一起复制”。

示范 3：改名 = 同目录内移动

```
$ mv ~/lab/ch2/note1.txt ~/lab/ch2/first-note.txt
$ ls ~/lab/ch2
a  first-note.txt  note2.txt
```

`mv` 没有独立的“`rename` 命令”——**改名就是移动到新名字**。想清楚这一点，你就理解 Linux 设计哲学了：一件事用一种工具完成。

2.3 删除: rm 和 rmdir

⚠ 危险警告: 删除不可恢复

Linux 没有回收站。**rm** 删掉的文件**找不回来**。还有两个与 Windows 最大的习惯差异: **命令行没有"复制 → 粘贴"两步**——**cp** 一步到位; **mv** **搬完原位置就消失**——它同时承担"移动"和"改名"。铁律: **先 ls 看清楚, 再 rm**; 实验只在 ~/lab 里做; **rm -rf** 敲之前默念一遍完整路径。

命令	作用
rm 文件	删文件
rm -r 目录	删目录及里面一切 (-r 递归)
rm -f 文件	强制删, 不询问
rmdir 目录	只删 空 目录 (有内容就报错, 相当于安全版)

示范 4: 安全地删除

```
$ ls ~/lab/ch2/a # 先看清里面有什么
b
```

```
$ rm -r ~/lab/ch2/a # 确认无误再删
```

"先 ls 后 rm"是运维的第一肌肉记忆。**rmdir** 删不掉非空目录时, 正是它在保护你。

2.4 易错点

坑 1: cp 不写目标会报错; cp 目标已存在会**直接覆盖**, 不提示。

坑 2: mv 目标已存在同样直接覆盖。

坑 3: mv 是"移动或改名", cp 是"复制一份"。mv 快 (同一磁盘只改记录), cp 慢 (真复制数据)。

✎ 章末作业

1. 用一条命令建出 ~/lab/ch2/x/y/z 三层目录。
2. 复制整个 ~/lab/ch2 到 ~/lab/ch2-backup, 验证里面文件都在。
3. 把 note2.txt 改名为 second-note.txt。
4. 尝试用 **rmdir** 删一个非空目录, 观察报错; 再用 **rm -r** 删掉它。

🎯 章末自测

1. 为什么说"改名就是移动"?
2. 判断: rm -rf 删错文件后可以用恢复软件找回。 (对/错)

✎ 对应练习册: 基础篇 1-4、1-5、1-6、1-7、1-8。

第 3 章 · 查看与搜索

💡 本册日志示范统一用自建文件 `~/lab/app.log`：先执行 `echo '2026-08-01 09:00:01 ERROR disk fail' >> ~/lab/app.log`（多敲几行不同内容）。系统日志路径取决于发行版、日志服务和配置；不要假定 `/var/log/syslog` 一定存在。先确认实际文件，或对 `systemd` 服务使用 `journalctl`。

本章目标：① 会用四种“看”命令按需查看文件；② 会用 `grep` 按内容搜、`find` 按名字找；③ 会 `wc` 计数、`diff` 对比；④ 理解 `tail -f` 的实时跟踪。

本章新命令：`cat less head tail wc file grep find diff`

3.1 四种“看”

命令	行为	适合
<code>cat</code>	一口气把整个文件全打印	小文件（几行到几十行）
<code>less</code>	翻页查看（空格下翻 / b 上翻 / q 退出 / /词 搜索）	大文件——less 一次只加载一屏，大文件不卡
<code>head</code>	看前 10 行（ <code>-n</code> 指定行数）	只看开头
<code>tail</code>	看后 10 行； <code>-f</code> 实时跟踪新增	看结尾；排错神器

示范 1：tail -f 实时跟踪日志

```
$ tail -f ~/lab/app.log
```

```
2026-08-01 09:00:01 ERROR disk fail
2026-08-01 09:00:02 WARN retrying in 3s
2026-08-01 09:00:03 INFO service recovered
```

`-f` = follow（跟随）：文件一有新增，立刻显示。排错标准动作：**开一个终端 `tail -f` 日志，另一个终端去复现问题**，报错会实时跳出来。按 `Ctrl+C` 退出。

示范 2：取文件中间某一段

```
$ head -n 120 ~/lab/app.log | tail -n 21
```

（第 100~120 行的内容）

`head` 先取前 120 行，交给 `tail` 取最后 21 行——合起来就是第 100~120 行。中间的竖线 `|` 叫“管道”（第 4 章主角），现在先感受一下。

3.2 按内容搜：grep

`grep` 的职责：**从文件里捞出含有某个词的行**。全称 Global Regular Expression Print（全局正则打印）。

写法	作用
<code>grep "error" 文件</code>	找含 <code>error</code> 的行
<code>grep -i error 文件</code>	忽略大小写（ <code>Error</code> 、 <code>ERROR</code> 都算）
<code>grep -r "关键词" 目录</code>	递归搜索整个目录
<code>grep -n error 文件</code>	显示行号
<code>grep -v "^#" 文件</code>	排除 <code>#</code> 开头的注释行（ <code>-v</code> 反选， <code>^</code> 表示行首）
<code>grep -c error 文件</code>	只数有多少行匹配

示范 3: grep 实战

```
$ grep -i "error" ~/lab/app.log
```

```
2026-08-01 09:00:01 ERROR disk fail
```

日志几万行，直接 cat 会刷屏刷到天荒地老。grep 先过滤，再看细节——这是读日志的标准姿势。

```
$ echo 'Port 22' > ~/lab/my.conf
$ echo '#PermitRootLogin no' >> ~/lab/my.conf
$ echo 'MaxSessions 10' >> ~/lab/my.conf
$ grep -v "^#" ~/lab/my.conf
```

```
Port 22
MaxSessions 10
```

-v = 反选（排除匹配行）。配置文件里通常有很多 # 注释行，用 grep -v "^#" 只看“真正生效的配置”。注意 ^# 的 ^ 表示“行首”——只排除 # 开头的行，正文里带 # 的不误伤。

3.3 按名字找: find

grep 搜内容，find 搜文件本身（名字、大小、时间）。

示范 4: find 实战

```
$ find ~ -name "*.log"
```

```
~/lab/app.log
~/lab/error.log
```

find 对 ~/lab 查找。find 的格式：**find 在哪找 + 按什么条件**。“*.log”中的 * 是通配模式。模式放在引号里时，shell 不会先展开它；**find 自己接收原样模式并递归匹配文件名**。去掉引号后，shell 可能先展开当前目录中的匹配项；当前目录没有匹配项时，行为还可能因 shell 选项而不同。因此 find 的通配模式通常要加引号。

```
$ find ~ -size +100M
```

```
~/videos/big-file.mp4
```

按大小找：+100M = 大于 100 兆。磁盘满的时候（第 7 章），用 find 揪出大文件。

3.4 计数与对比: wc、file、diff

命令	作用
wc -l 文件	数行数 (-w 词数, -c 字节数)
file 文件	看文件真实类型（不看名字，看内容）
diff 文件1 文件2	对比两个文件的差异（验证操作正确性的利器）

示范 5: 用 diff 验证复制无误

```
$ diff -r ~/lab/ch2 ~/lab/ch2-backup
```

（没有任何输出）

diff 没输出 = 两个目录完全一致。以后做完复制、备份、解压，都可以用 diff -r 验证“一个不多一个不少”。

3.5 易错点

坑 1: cat 大文件会刷屏，机器卡住几秒都正常——大文件用 less。

坑 2: grep 的搜索词建议加引号: grep "my log", 否则空格和特殊符号会捣乱。

坑 3: grep 搜内容 vs find 找文件，面试常考，别混。

章末作业

1. 用 less 打开 ~/lab/app.log, 练习 空格 / b / q / 搜索。
2. 用 grep -i 在 ~/lab/app.log 里找 "error", 用 grep -c 数一共几行。
3. 用 find 找出家目录下所有 .txt 文件。
4. 复制一个目录后用 diff -r 验证两边一致，再改动一个文件后看 diff 输出长什么样。

章末自测

1. 想看一个 5 万行日志的最新报错，应该用哪条命令？（A. cat B. less C. tail -f）
2. 判断：grep 是按文件名找文件，find 是按内容找文件。（对/错）

 对应练习册：基础篇 2-3、2-4、2-5、2-6、2-7；阶段 5 的 5-1、5-2、5-3；进阶篇场景 1。

第 4 章 • 管道与重定向

本章目标：① 理解管道 | 的流水线思想；② 区分 >（覆盖）与 >>（追加）；③ 会 sort / uniq 配合管道；④ awk 入门：按列取数据。

本章新命令：sort uniq awk（符号：| > >> 2>&1）

4.1 管道：把命令串成流水线

每个 Linux 命令都像一道**工序**：吃进数据，吐出结果。**管道 |**就是把前一道工序的"吐出"，直接喂给下一道工序的"吃进"。

📌 管道的哲学：**每个命令只做一件小事，用管道把小工具组合成大功能**。这就是 Linux 力量的来源。

示范 1：三道工序接力

```
$ grep -i "error" ~/lab/app.log | wc -l
```

```
3 （你自建了几行就是几）
```

grep 捞出行 → 交给 wc -l 数行数。**不落地、不存中间文件**，这就是管道的好处。

4.2 重定向：改变输出的流向

符号	作用
>	把输出 覆盖写入 文件（文件原有内容会 清空 ）
>>	把输出 追加 到文件末尾
2>&1	把"错误输出"合并到"正常输出"一起去同一个地方

示范 2：> 和 >> 的差别（亲手验证）

```
$ echo "第一行" > ~/lab/test.txt
$ echo "第二行" > ~/lab/test.txt
$ cat ~/lab/test.txt
```

```
第二行
```

```
$ echo "第三行" >> ~/lab/test.txt
$ cat ~/lab/test.txt
```

```
第二行
第三行
```

两次 > 之后文件里只剩"第二行"——> **每次都会清空重写**。>> 则乖乖排在后面。这个实验一定要亲手敲一遍，比看十遍都记得牢。

易错点：往重要文件写内容时，**先备份再 >**。一条 echo x > 配置.conf 会把原配置清得干干净净。

4.3 sort 和 uniq

sort 排序; uniq 去重 (只对相邻重复行生效, 所以标准用法是先 sort 再 uniq) ; -c 顺便计数。

示范 3: sort + uniq -c

```
$ echo 'bob' > ~/lab/names.txt; echo 'alice' >> ~/lab/names.txt; echo 'alice' >>
~/lab/names.txt; echo 'carol' >> ~/lab/names.txt; echo 'carol' >> ~/lab/names.txt; echo
'carol' >> ~/lab/names.txt # 先自造名单文件 (alice×2、bob×1、carol×3)
```

```
$ sort ~/lab/names.txt | uniq -c
```

```
2 alice
1 bob
3 carol
```

sort 把相同的挤到一起, uniq -c 才能正确统计。名字后的数字是出现次数。这是“分组计数”的标准套路。

4.4 awk 入门: 按列取数据

awk 是文本处理利器, 入门只需要记住一件事: 按空格把一行切成几列, \$1 是第一列, \$2 是第二列.....

示范 4: 取日志的第一列

```
$ echo "Aug 16 09:30 ubuntu kernel: boot ok" | awk '{print $1}'
```

```
Aug
```

```
$ echo 'Aug 09:30:01 kernel: boot ok' > ~/lab/log.txt; echo 'Sep 14:02:55 kernel: shutdown ok'
>> ~/lab/log.txt # 先自造两行日志
```

```
$ awk '{print $1, $3}' ~/lab/log.txt
```

```
Aug 09:30
Sep 14:02
```

awk 的固定格式: awk '{print 你想取的列}'。单引号里的内容是一小段“取列指令”。\$0 表示整行。

4.5 日志分析标准套路 (本章精华)

↑ 提取 → 排序 → 去重计数 → 倒序 → 取前 N。五步连环, 日志统计题一招通吃。

示范 5: 统计访问日志里出现最多的 IP 前 5 名

```
$ printf '192.168.1.10 abc\n203.0.113.9 xyz\n' >> ~/lab/access.log # 先自造日志, 多敲几行不同
IP 的 (printf = 增强版 echo, 首次出现: 它认 \n 等转义字符, \n 表示换行)
$ awk '{print $1}' ~/lab/access.log | sort | uniq -c | sort -rn | head -5
```

```
2 192.168.1.10
1 203.0.113.9 (示例—你的数字按实际写入为准)
```

逐环解释: awk 取 IP 列 → sort 排序 (相同的挨一起) → uniq -c 计数 → sort -rn (r=倒序, n=按数字) → head -5 取前五。把这个管道背下来, 它就是你的第一个“组合拳”。

4.6 易错点

坑 1: > 会清空文件。先想清楚"我要覆盖还是追加"。

坑 2: 管道只传递"正常输出", 错误信息不走管道 (所以 grep 没结果时不报错, 就是没匹配)。

坑 3: awk 的单引号不能换成双引号。

章末作业

1. 把 echo 输出分别用 > 和 >> 写 3 次, cat 验证差别 (照示范 2 做)。
2. 造一个多行文本, 用 sort | uniq -c 统计每个词出现次数。
3. 用 awk 取出 /etc/passwd 的第一列 (用户名)。
4. 把示范 5 的管道改造成: 统计自己日志里出现最多的 3 个词。

章末自测

1. 管道 | 和重定向 > 的本质区别是什么?
2. 判断: uniq 可以直接对乱序文件正确去重。 (对/错)

 对应练习册: 基础篇 2-4、2-8; 阶段 5 的 5-4、5-5。

5.4 chmod: 改权限 (含数字算法原理)

两种写法:

写法	示例	说明
符号法	chmod u+x 文件	u=属主、g=组、o=其他人、a=全部; +=加、-=减。只动你指定位
数字法	chmod 755 文件	一次覆盖全部九位

⚠ **数字算法原理** (为什么 r=4、w=2、x=1): 权限位本质是三个开关, 开=1 关=0。r=4(100)、w=2(010)、x=1(001), 相加即组合: rwx=7、rw-=6、r-x=5、r--=4。755 = 属主 rwx(7) + 属组 r-x(5) + 其他人 r-x(5)。644 (文件最常见) = rw-r--r--。

示范 2: chmod 实战

```
$ touch ~/lab/script.sh # 先自造目标文件 (示范自足, 跳章也能跑)
$ chmod u+x ~/lab/script.sh
$ ls -l ~/lab/script.sh
```

```
-rwxr--r-- 1 <用户名> <用户名> 0 Aug 16 10:00 script.sh (示例输出, 实际以本机为准)
```

u+x 只给属主加执行位: rw- 变 rwx, 其余不变。**符号法动一位, 数字法全覆盖**——两种方法目标一致时结果相同 (如 chmod u+x 与 chmod 744 都得到 744)。

5.5 chown / chgrp: 换主人

示范 3: 换属主属组

```
$ touch ~/lab/shared.txt # 先自造一个测试文件
$ sudo chown alice:alice ~/lab/shared.txt
```

chown = change owner。格式"用户:组"。chgrp 只换组不换人。注意: 换了主人后, 原来的用户就按"其他人"或"属组"的权限位来判了 (前提: 他不在这文件的属组里)。

5.6 其他用户管理命令

命令	作用
id 用户名	看用户的 UID、GID 和所属组 (比 whoami 详细)
userdel -r 用户名	删除用户 (-r 连家目录一起删)
groupadd 组名	创建组 (做共享目录的第一步)
last	看最近谁登录过这台机器

⚠ 危险警告: chmod 777

chmod 777 文件 = 属主、属组、其他人都能读写执行——相当于把家门钥匙插在锁上。遇到"权限问题先 777 再说"的坏习惯, 面试和实战都会翻车。正确姿势: **搞清楚谁需要什么权限, 给最小权限。**

5.7 易错点

坑 1: 新建用户不加 -m, 没有家目录, 登录会各种奇怪。

坑 2: 删除是看**目录**写权限, 不是文件本身权限 (练习册 1-7、3-8 会验证这个反直觉结论)。

坑 3: **chmod -R** 递归改整个目录很危险, 先想清楚再敲。

章末作业

1. 创建 alice、bob 两个用户并设密码，用 groups 验证各自所在组。
2. 把 alice 加入 sudo 组 (usermod -aG) ，验证她能 sudo 了。
3. 建一个文件，分别用符号法和数字法把它改成 755，两次都 ls -l 验证。
4. 用 chown 把文件换给 bob，再用 alice 身份试试能不能读它。

章末自测

1. 权限串 -rwxr-xr-- 中，"其他人"的权限是什么？数字法是几？
2. 判断：sudo 是切换成 root 用户，su 是借 root 权限执行一条命令。（对/错）

 对应练习册：基础篇 3-1 ~ 3-8。

第 6 章 • 进程管理

本章目标：① 看懂 ps aux 各列；② 会找进程（pgrep / ps+grep）；③ 理解"优雅杀 vs 强杀"；④ 会前后台切换与 nohup；⑤ 会用 fuser 定位占端口的进程。

本章新命令：ps top htop pgrep kill killall jobs bg fg nohup fuser

6.1 进程是什么

进程 = 正在运行的程序。同一个程序可以开多个进程（开两个终端就是两个进程）。每个进程有一个编号 **PID**（工号），管理进程全靠它。

示范 1：ps aux 各列解读

```
$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.1	163904	12340	?	Ss	08:00	0:01	/sbin/init
<用户名>	1234	0.5	2.3	456704	234560	pts/0	S+	09:10	0:02	python3 app.py

（示例输出，实际以本机为准）

重点看四列：**USER**=谁启动的、**PID**=工号、**%CPU/%MEM**=吃多少资源（机器卡了先看这两列谁最高）、**COMMAND**=是什么程序。**aux** 三个字母是组合选项（a=所有用户、u=带用户信息、x=含无终端的）。

top 是实时版的 ps（每 3 秒刷新，按 q 退出）；**htop** 是彩色增强版（可能要先 `sudo apt install htop`，第 9 章会教）。

6.2 找进程

示范 2：三种找法

```
$ ps aux | grep nginx # 通用法：ps 全量 + grep 过滤
$ pgrep nginx # 快捷法：直接输出 PID
```

```
1234
5678 （示例 PID：实际是你的进程号）
```

pgrep = process grep，一步到位拿 PID。ps aux | grep 是面试必考的组合拳，两者都要会。

6.3 结束进程：优雅 vs 强杀

kill 不是"杀死"，是"发信号"。默认发的是 SIGTERM (15)："请你退出"。程序收到后可以打扫房间——写日志、存数据——然后体面离开。加 **-9** 是 SIGKILL："当场击毙"，由内核直接处决，程序**没有任何清理机会**。

示范 3：亲眼看见"优雅退出"（先建一个会写日志的小程序）

本示范用到了 `cat > 文件 << 'EOF'` 这个写法：把下面整段内容直接"喂"进文件（直到出现 EOF 为止），不用一行行 echo——这是快速造多行文件的标准姿势（heredoc，读作"此文档"）；EOF 是约定的结束标记，前后各放一个。

```
$ cat > ~/lab/run.sh << 'EOF'
#!/bin/bash
trap 'echo "$(date +%T) 收到退出信号，清理中..." >> ~/lab/run.log; exit 0' TERM
while true; do sleep 1; done
EOF
$ chmod +x ~/lab/run.sh
$ cd ~/lab && ./run.sh &
MY_PID=$! # $! = 刚启动的后台任务真实 PID（不靠猜）
$ ps -p $MY_PID # 先核对：确实是我们启动的 ./run.sh
```

```
PID TTY TIME CMD
4242 pts/0 00:00:00 /bin/bash ./run.sh （示例 PID，以你 ps 实际看到的为准）
```

```
$ kill $MY_PID
$ cat ~/lab/run.log
```

```
09:30:15 收到退出信号，清理中...
```

trap 那行 = 程序安装了"信号处理器"：收到 TERM 就写一行日志再退出。**kill（优雅）→ 日志里留下了告别记录**。现在再用 `kill -9` 试一次：日志里**不会**有新记录——程序连写日志的机会都没有。。

📌 细节提示：PID 和日志时间是动态的——`$!` 拿到的是你的实际 PID；`pgrep` 可能匹配多个同名进程，所以用 `$!` 保存、用 `ps -p` 核对，**只动自己的实验进程**。run.log 写进 `~/lab/`，从任何目录启动都不乱跑。

killall = 按**名字**杀进程（不用查 PID）：`killall nginx`。名字匹配到多个进程会全部杀掉，先 **pgrep** 确认再动手。

⚠ 危险警告：kill -9 滥用

对数据库、推理服务直接 **kill -9** 可能造成数据损坏。**-9 永远是最后手段**：先试普通 kill → 没反应再观察 → 仍无效才 -9。养成习惯，面试和实战都加分。

6.4 前后台与 nohup

命令	作用
命令 &	末尾加 & = 放后台运行，终端继续给你用
Ctrl + Z	把前台程序"挂起"（暂停）
jobs	看后台任务清单
fg %1 / bg %1	把 1 号任务拉回前台 / 放后台继续跑
nohup 命令 &	后台运行 + 关掉终端也不死 （no hang up）

示范 4: nohup 挂后台长任务

```
$ echo 'import time; time.sleep(3600)' > ~/lab/train.py # 先自造一个"跑一小时"的模拟训练脚本
```

```
$ nohup python3 ~/lab/train.py > ~/lab/train.log 2>&1 &  
TRAIN_PID=$! # 记下 PID, 练完 kill $TRAIN_PID 清理, 别留长任务占机器
```

拆开看: nohup (关终端不死) + > ~/lab/train.log (输出进文件, 和脚本同目录、路径一致) + 2>&1 (错误也进同文件) + & (后台)。长任务标准姿势。SSH 断开后任务照样跑。练完 kill \$TRAIN_PID 清理。

6.5 fuser: 谁占着这个端口/文件

fuser 回答一个高频问题: "谁在用这个东西"。端口报"Address already in use"时就用它揪人。

示范 5: fuser -v

```
$ python3 -m http.server 8080 & # 先造一个可控监听 (& = 放后台)  
SRV_PID=$! # 记下它的 PID, 验证完要清理  
$ fuser -v 8080/tcp
```

USER	PID	ACCESS	COMMAND
8080/tcp:	<用户名>	<实际 PID>	F.... python3 (示例 PID, 以你实际输出为准)

```
$ kill $SRV_PID # 验证完清理, 别留后台服务  
$ fuser -v 8080/tcp # 没输出 = 端口已释放
```

-v = 详细。输出告诉你: 8080 端口被 <实际 PID> 的 python3 占着 (PID 是动态的, 以本机实际输出为准)。fuser 属于 psmisc 软件包, 没装就先 sudo apt install psmisc。也可以查文件: fuser -v 某个文件。

6.6 易错点

- 坑 1: ps 不带 aux 只显示自己的进程——"找不到进程"先检查是不是忘了 aux。
- 坑 2: kill 之后用 ps 再确认一次, 有些进程要几秒才消失。
- 坑 3: & 忘打, 程序霸占终端——Ctrl+C 或 Ctrl+Z 先救场。

✎ 章末作业

1. 用 ps aux | grep 找到你的终端进程, 读出它的 PID 和 %CPU。
2. 后台跑一个 sleep 300, 用 jobs 看它, 再用 kill 优雅结束它。
3. 重做示范 3 的完整实验: kill 和 kill -9 各来一次, 对比 run.log。
4. 用 nohup 挂一个后台任务, 关掉终端重新打开, 验证它还在跑。

🎯 章末自测

1. kill 和 kill -9 的区别是什么? 为什么 -9 是最后手段?
2. 判断: nohup 的作用是"忽略错误输出"。(对/错)

✎ 对应练习册: 基础篇 4-1 ~ 4-5; 进阶篇场景 1、4 (fuser 用于定位占端口进程)。

第 7 章 · 磁盘与文件系统

本章目标：① 会用 `df` 看整体、`du` 看明细；② 理解"挂载"；③ 会造测试大文件（`fallocate` / `dd`）；④ 看懂磁盘满的排查套路。

本章新命令：`df du lsblk mount fallocate dd mkfs`

⚠ **mount / mkfs 是危险操作，别急着动手：**完整演练流程见《Linux 练习册·进阶篇》场景 2 搭建手册（在镜像文件里练，安全）。先记住两个词义：**mount** = 把设备接到目录树上；**mkfs** = 格式化——**绝对不对真磁盘乱用。**

7.1 磁盘、分区与挂载

比喻：**硬盘=仓库，分区=仓库隔间，挂载=把隔间接进目录树。**Linux 没有 Windows 的 C 盘 D 盘——新硬盘必须"挂"到某个目录（如 `/mnt/data`），这个目录就成了它的入口。

顺带把一词说清：这种"分区 + 目录树"的组合就叫 **文件系统**（`filesystem`）——你以后听到的 `ext4`、`xfs`、"新硬盘要先格式化"，说的都是它。

示范 1: `df -h` 看整体

```
$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	500G	120G	355G	26%	/ （示例输出，实际以本机为准）

-h = human（人类可读，自动换算 G/M）。**Use%** 是核心：超过 80% 要警惕，100% 服务会挂。**Mounted on** 列就是"挂载点"。

示范 2: `du -sh` 看谁占了空间

```
$ du -sh ~/* # 明确指定家目录；* 不展开隐藏文件（.开头），想看全部先 ls -a ~
```

```
4.0K lab
2.3G videos
1.1G Downloads （示例输出，实际以本机为准）
```

`df` 告诉你"仓库满没满"，`du` 告诉你"哪个箱子最占地方"。磁盘满的排查套路：**`df -h` 定位满的分区 → `du` 逐层下钻 → `find -size +100M` 揪出大文件（第 3 章）→ 确认无用后删除或归档。**

易错：`df` 和 `du` 的数字对不上是经典现象——常见原因是"文件被删了但还有进程占着"（实战中常见，先记住这个结论）。

挂载与卸载速览（`mount` / `umount`）：`sudo mount /dev/sdb1 /mnt/data` 把分区接到目录 → `df -h /mnt/data` 验证（列表里能看到即成功）→ `sudo umount /mnt/data` 卸载（**拼写没有 n**，别写成 `unmount`）。卸载报 **target is busy** 时：先 `cd` 离开该目录、结束占用它的程序，或 `ls -lsof /mnt/data` 查是谁在用。完整演练见《Linux 练习册·进阶篇》场景 2 搭建手册（镜像文件里练，安全）。

7.2 造测试文件: fallocate 和 dd

命令	说明
<code>fallocate -l 100M 文件</code>	瞬间造一个 100M 大文件 (秒完成, 练习用首选)
<code>dd if=/dev/zero of=文件 bs=1M count=100</code>	造 100M 文件 (真写入, 可造镜像文件)

示范 3: fallocate 快速造大文件

```
$ df -h ~ # 先确认磁盘空间够 (本章作业 3 也造 100M, 口径一致)
$ fallocate -l 100M ~/lab/big.img
$ ls -lh ~/lab/big.img
```

```
-rw-r--r-- 1 <用户名> <用户名> 100M Aug 16 11:00 big.img (示例输出, 实际以本机为准)
```

一秒造出 100M 文件, 练习"磁盘占用"场景的原料。容量按你的磁盘空间调整 (50M / 100M 都行), 先 `df -h` 确认够再动手。dd 的 `if`=输入文件 (`/dev/zero` 是无限零的设备)、`of`=输出文件、`bs`=块大小、`count`=块数。

⚠ 危险警告: dd 的 of= 写错 = 灾难

dd `if=/dev/zero of=/dev/sda` 会直接覆盖整块硬盘。用 dd 前必须先 `lsblk` 确认设备名; `of`= 永远不要指向真实硬盘/分区。练习只用普通文件名。

7.3 内存与负载: free 和 uptime

磁盘用 `df/du` 看, 内存用 `free`、负载用 `uptime`——资源占用三兄弟齐了。

示范 4: free 与 uptime

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	31G	2.1G	26G	120M	2.3G	28G
Swap:	2.0G	0B	2.0G			

(示例输出, 实际以本机为准)

`free -h` 看内存: `total`=总内存、`used`=已用、`available`=真正可用 (含缓存可释放部分)。Linux 的"内存看着满"常常是缓存, 先看 `available` 再下结论。

```
$ uptime
```

```
11:30:15 up 2 days, 3:14, 2 users, load average: 0.15, 0.10, 0.08 (示例输出, 实际以本机为准)
```

`uptime` 看两件事: 开机多久 + 三个负载值 (最近 1/5/15 分钟平均)。负载是"排队等 CPU 的任务数", 持续超过 CPU 核数说明机器忙。

7.4 易错点

坑 1: `mkfs` 是"格式化" (清空建文件系统), 练习机上永远别碰真实分区。

坑 2: `du` 不带 `-sh` 会列出每个子目录, 输出刷屏; 先加 `-s` (汇总)。

坑 3: `df` 忘记 `-h`, 数字长得没法读。

章末作业

1. `df -h` 找出根分区使用率，记下来。
2. 先 `ls -a ~` 确认家目录里有哪些项（含隐藏），再用 `du -sh ~/*` 找出最占空间的 3 个东西。
3. `fallocate` 造一个 100M 文件，`ls -lh` 验证大小。
4. `lsblk` 看你机器有几块硬盘、几个分区，对照 `df` 的输出找对应关系。

章末自测

1. `df` 和 `du` 的区别是什么？排查磁盘满时先用哪个？
2. 判断：Linux 的新硬盘会自动出现 D 盘、E 盘。（对/错）

 对应练习册：基础篇 4-5；进阶篇场景 2（磁盘满应急）。

第 8 章 · 网络命令

本章目标：① 会查 IP、网关 (ip / ip route)；② 会用 ping、ss、nc、curl 做基础排查；③ 会 ssh / scp 远程操作；④ 掌握"主机→端口→进程→日志"四层排查法。

本章新命令：ip ip route ping ss curl wget nc traceroute nslookup ssh scp

外部条件说明：nc / traceroute / nslookup 通常需 apt 安装 (netcat-openbsd、traceroute、dnsutils)；ssh / scp 需要远程主机。主线 (ip / ping / ss / curl / wget) 本地可完成；扩展命令只演示用法，装不上不影响主线。

8.1 地址与端口

IP = 门牌号 (机器在网络里的地址)，**端口 = 门上的窗口号** (一台机器上不同服务各开一个窗口：网页 80、SSH 22、HTTPS 443)。“连不上服务”= 要么门牌找不到，要么窗口没开。

示范 1：查本机 IP 和网关

```
$ ip addr
```

```
2: ens33: inet 192.168.1.10/24 ... (示例输出，实际以本机为准)
```

在 inet 后面找本机 IP (192.168.1.10)。/24 表示子网掩码，暂时不用深究。

```
$ ip route
```

```
default via 192.168.1.1 dev ens33 (示例输出，实际以本机为准)
```

ip route (查看路由表)：default via 后面就是你的**默认网关** (一般就是路由器地址)。排查“上不了网”从这里开始。

8.2 排查三件套：ping、ss、nc

命令	作用
ping -c 4 目标	测通不通 (-c 限定发 4 个包就停)
ss -tlnp	看本机哪些端口在 监听 (t=TCP、l=监听、n=数字、p=程序名)
nc -zv 主机 端口	探测对方端口开没开 (-z=只探测不传数据、-v=详细)

示范 2：ss 和 nc 实战

```
$ ss -tlnp
```

```
LISTEN 0 128 0.0.0.0:22 users:(("sshd",pid=<实际 PID>,fd=3)) (示例输出：端口、进程名、PID 均以本机为准)
```

22 端口 (SSH 服务) 在监听，程序是 sshd。谁在哪个端口一目了然。注：-p 显示的进程名需要权限——普通用户看不到别人的进程名属正常，用 **sudo ss -tlnp** 可看全部。

```
$ nc -zv localhost 22
```

```
Connection to localhost 22 port [tcp/ssh] succeeded!
```

```
$ nc -zv localhost 9999
```

```
nc: connect to localhost port 9999 (tcp) failed: Connection refused
```

“在听”和“能连上”是两回事：ss 看**本机**谁在听；nc 从**访问者角度**测能不能连。第一次用 nc 若提示找不到命令：**sudo apt install netcat-openbsd** (第 9 章会教你 apt)。

8.3 取数据: curl 和 wget (本地可复现版)

本节全部用本地服务演示, 不需要外网; 文末附一个“联网扩展”例子 (无网络直接跳过, 不影响主线验收)。

示范 3: 先用 python3 -m http.server 起一个本地网页服务

```
$ mkdir -p ~/lab && cd ~/lab && python3 -m http.server 8000 &
```

Serving HTTP on 0.0.0.0 port 8000 ... (以你机器实际输出为准)

python3 -m http.server = 把当前目录变成一个网页服务, 8000 是端口; 结尾 & 让它后台跑不占终端。用完收尾: jobs 找到它, kill 掉 (或 kill %1)。

示范 4: curl 成功请求 + 404 失败 (本地稳定可复现)

```
$ curl -I http://localhost:8000/
```

HTTP/1.0 200 OK

Server: SimpleHTTP/0.6 Python/3.x.x (以你机器实际输出为准)

```
$ curl -I http://localhost:8000/不存在.html
```

HTTP/1.0 404 File not found (以你机器实际输出为准)

curl = 发 HTTP 请求拿数据 (调 API、测网站、看返回)。200 = 成功; 404 = 服务器上没这个文件。常用选项: -I 只看响应头、-o 文件 存到文件。

示范 5: wget 本地演示 (下载成功 + 404 失败)

```
$ echo test > hello.txt
```

```
$ wget -O downloaded-hello.txt http://localhost:8000/hello.txt
```

... 'downloaded-hello.txt' saved [5/5] (成功: 以实际输出为准)

```
$ wget http://localhost:8000/不存在.html
```

HTTP request sent, awaiting response... 404 File not found

ERROR 404: File not found. (失败: 退出码非 0, 以实际输出为准)

wget = 下载文件。-O 指定“保存成什么名字”——这里源文件 hello.txt 和下载目标在同一目录, 不指定 -O 的话 wget 会改名存成 hello.txt.1 (避免覆盖已存在的同名文件)。404 时 wget 会打印 ERROR 404 提示——注意: Ubuntu 22.04 实测 wget 对 404 的退出码仍是 0, 所以脚本里判断下载成败别只依赖 wget 退出码, 看输出提示、或用 curl --fail (失败时退出码非 0) 更可靠。演示完收尾: kill %1 停掉本地服务、rm hello.txt downloaded-hello.txt 清理。

易错: curl 测“服务通不通”, wget 下“文件”。URL 里的 & 会被 shell 当后台符号处理, 带 & 的长 URL 必须加引号。

🌐 联网扩展 (可跳过, 不影响主线): 带参数的公开 API 示例——curl 'https://api.open-meteo.com/v1/forecast?latitude=30.25&longitude=120.17¤t_weather=true' (天气 API, 免注册)。返回 JSON, 值随时变化; 需要网络, 连不上就跳过——本节所有验收只要求本地部分。

8.4 远程操作：ssh 和 scp

示范 6：ssh 登录 + scp 传文件 🌐

```
$ ssh 用户名@192.168.1.10 # 远程登录（-p 可指定端口）  
$ scp ~/lab/file.txt 用户名@192.168.1.10:/tmp/ # 把文件拷到远程机器
```

ssh = secure shell，远程登录后就像坐在那台机器前一样敲命令。**scp** = 通过 SSH 跨机器拷贝文件。练习条件：被连的机器要先装 openssh-server (Ubuntu: `sudo apt install openssh-server`)。

补充两个命令：**traceroute** 目标 = 显示数据包沿途经过的每一跳（定位“堵在哪一段”）；**nslookup** 域名 = 域名查 IP（DNS 解析排查，如 `nslookup deepseek.com`）。

8.5 四层排查法（本章精华）

↑ 服务连不上时，按证据分层排查，不能由一次 ping 直接下结论：

- ① **回环层**：`ping -c 4 127.0.0.1`，只验证本机回环和网络栈；
- ② **本机/网关层**：`ip addr` 看地址，`ip route` 找默认网关，再测试网关；
- ③ **目标主机层**：`ping` 目标 IP；ICMP 可能被防火墙禁用，ping 不通不等于机器关机；
- ④ **端口/进程/应用层**：用 `nc -zv` 或 `curl` 测端口和应用，再用 `ss`、`ps`、日志定位服务。

这条路线和练习册进阶篇场景 3 的题目一一对应，做完题回来再读一遍，体会更深。

8.6 易错点

坑 1：ping 不通 ≠ 对方没开机——对方可能禁 ping（防火墙），但网页服务正常。继续用 nc 测端口。

坑 2：ss 是新一代 netstat，旧教程里的 netstat 用 ss 替代。

坑 3：curl 的 URL 带 & 参数时，URL 要加引号。

✍ 章末作业

1. 用 `ip addr` 找出本机 IP，用 `ip route` 找出网关地址。
2. 用 `ss -tlnp` 列出本机所有监听端口，说出每个端口是干什么的（22 号至少认识）。
3. 用 `nc -zv` 测 localhost 的 22 端口和一个不存在的端口，对比输出。
4. 用 `curl` 请求一个公开网址（如 `example.com`），用 `-I` 只看响应头。

🎯 章末自测

1. 默写四层排查法（主机→端口→进程→日志）每一步用什么命令。
2. 判断：ping 不通说明对方机器一定关机了。（对/错）

✍ 对应练习册：进阶篇场景 3（全部题目）；场景 1 的 ss 部分。

第 9 章 • apt 包管理

本章目标：① 理解包管理器为什么存在；② 会用 apt 装/卸/搜软件；③ 看懂常见的锁文件报错。

本章新命令：apt update apt install apt remove apt search apt upgrade apt list dpkg

9.1 包管理器：应用商店 + 自动管家

软件之间互相依赖（A 需要 B，B 需要 C）。手动装软件要自己解决依赖地狱；**apt = 应用商店 + 自动管家**：你只说“我要装 htop”，它自动把依赖全拉齐、装好、登记在册。

命令	作用
sudo apt update	刷新货架（更新软件列表，本身不装任何东西）
sudo apt install 软件	安装
sudo apt search 关键词	搜货架
sudo apt remove 软件	卸载
sudo apt upgrade	把已装的都升级到最新
apt list --installed	看已装清单（常配 grep 过滤）

📌 外部条件与四种状态：apt 需要**网络 + 软件源 + sudo 权限 + dpkg 状态正常**。①**已安装**：apt 提示“已是最新版本”；②**未安装**：先 sudo apt update 刷新再装；③**无网络**：update 会超时/失败，主线概念不受影响，安装题记为环境扩展；④**被锁定**：报 Could not get lock，等另一个 apt 结束（见 9.3）。**安装后的软件不能当默认已有**——每台新机器都要先装。

示范 1：完整装一个软件 🌐

```
$ sudo apt update
```

```
Get:1 http://archive.ubuntu.com ... [1,234 kB] （示例输出，实际以本机为准）
```

```
$ apt search htop
```

```
htop/<发行版代号> 3.3.0-4 amd64
  interactive processes viewer （示例输出：Ubuntu 版本不同会显示不同代号（22.04=jammy、
  24.04=noble），以本机实际显示为准）
```

```
$ sudo apt install htop
```

```
$ htop
```

四步标准流程：**update 刷新** → **search 确认名字** → **install 安装** → **直接运行验证**。以后想装任何软件，照这个流程走。

9.2 经典报错解读

示范 2：锁文件报错

```
$ sudo apt install htop
```

```
E: Could not get lock /var/lib/dpkg/lock - open (11: Resource temporarily unavailable)
```

原因：另一个 apt 正在跑（比如系统自动更新），或者上次 apt 被 Ctrl+C 打断没清理。**锁**=防止两个 apt 同时改系统。解决：等它跑完；确认没有别的 apt 后重启一次系统锁会自动释放。

底层补充：**dpkg** 是 apt 底层的“装机工”，apt 最终通过它干活。日常用 apt 就够，知道 dpkg 的存在即可——报错里见到 dpkg 字样不用慌。

易错点： update 和 upgrade 天天被搞混——**update 刷新列表（几乎每次装东西前都做），upgrade 真正升级软件。**装新软件只需要 update + install。

章末作业

1. 用 apt search 找一个你感兴趣的小软件（如 cowsay、neofetch），装它并运行。
2. apt list --installed 配合 grep，确认 htop 是否已安装。
3. 用 apt remove 卸掉刚装的小软件，再 search 确认它没了。
4.  复现锁报错：终端 1 运行 `sudo apt install linux-firmware`（体积较大、下载需片刻，够时间复现），趁它下载时在终端 2 运行 `sudo apt install htop`——观察终端 2 的锁报错。实验完终端 1 按 Ctrl+C 中断，并清理残留：`sudo dpkg --configure -a`。

章末自测

1. apt update 和 apt upgrade 的区别？
2. 判断：apt 报"Could not get lock"说明磁盘满了。（对/错）

 对应练习册：基础篇 4-7（htop 安装）；进阶篇各场景的 nc、htop 安装。

第 10 章 • systemctl 与服务

本章目标：① 理解 systemd 和"服务"；② 会用 systemctl 管理服务全流程；③ 会用 journalctl 看服务日志。

本章新命令：systemctl journalctl

10.1 systemd：开机大管家

服务 = 常驻后台的程序（nginx 网页服务、sshd 远程登录服务、docker 容器引擎.....）。它们没人盯着也要 7×24 运行。**systemd 就是大管家**：开机时按顺序拉起所有服务，运行时盯着它们，崩了拉起来。

systemctl 是你和管家对话的命令。它管理的每个服务都有一份"档案"（unit 文件），记录启动命令、依赖、重启策略。

示范 1：服务管理全流程 🌐

```
$ systemctl status ssh
```

```
• ssh.service - OpenBSD Secure Shell server
  Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled)
  Active: active (running) since Fri 2026-08-15 08:00:01 CST; 1 day ago
  Main PID: 890 (sshd) （示例时间和 PID，以你机器实际输出为准）
```

```
$ sudo systemctl restart ssh
$ systemctl status ssh
```

status 输出三行是核心：**Loaded**=档案在哪（enabled=开机自启）；**Active**=现在是活是死（running=活着，failed=挂了，这里是排查第一眼）；**Main PID**=主进程工号。

命令	作用
systemctl start / stop 服务	启动 / 停止（只对这一次生效）
systemctl restart / reload 服务	重启 / 不中断重读配置
systemctl status 服务	看状态
systemctl enable / disable 服务	设/取消 开机自启

易错点：start ≠ enable。start 是"现在启动"；enable 是"开机时自动启动"。装了服务忘记 enable，重启机器后服务不回来——经典坑。改完配置文件后要 **restart**（或 reload）才生效。

10.2 journalctl：服务的日记本

示范 2：journalctl 实战

```
$ journalctl -u ssh # 只看 ssh 服务的日志
```

```
$ journalctl -u ssh -f # 实时跟踪
```

```
$ journalctl -p err # 只看错误级别以上的
```

服务挂了的三板斧：**systemctl status 看死活** → **journalctl -u 服务名 看日志** → **journalctl -p err 只看报错**。和《日志报错解读手册》配合使用。

📌 外部条件：本章用 ssh 服务做演示对象——Ubuntu 桌面版可能没装 openssh-server（先 `sudo apt install openssh-server`，或用系统已有的任意服务代替，如 cron、systemd-resolved）。**停止正在用的 SSH 服务会断掉你的远程会话**：实验请在本地终端做，远程时改用"重启"而非"停止"，并留好本地终端恢复方式。WSL / 容器环境 systemd 可能受限，不能做时记为环境限制，不影响主线概念。

章末作业

1. `systemctl status ssh`，逐行解释 Loaded、Active、Main PID 的含义。
2. 对 ssh 服务完整走一遍 start/stop/restart，每次都用 status 验证。
3. `journalctl -u ssh` 看它的日志，再 `-p err` 过滤只看错误。
4. 查一下 ssh 是否 enabled（开机自启），说出判断依据。

章末自测

1. start 和 enable 的区别？装了个服务想“重启机器后自动运行”该敲哪个？
2. 服务挂了，排查的第一步该做什么？（A. 先 `systemctl status` 看死活，再 `journalctl` 看日志 B. 先 `journalctl` 看日志，再 status C. 只看 status 就够 D. 只看日志就够）

 对应练习册：基础篇 4-6；进阶篇场景 4（开机自启）。

第 11 章 · 环境变量

本章目标：① 理解环境变量是"全局公告栏"；② 明白 PATH 决定"命令找不找得到"；③ 会 export、改 rc 文件、source；④ 会 alias 起外号。

本章新命令：env echo export source alias (文件： ~/.bashrc)

11.1 环境变量：贴在墙上的公告

每个终端都有一面"公告栏"，上面贴着各种公告：**HOME**=你家在哪、**PATH**=去哪找命令、**USER**=你是谁。程序跑起来之前会先看公告栏。**env** 能看全部公告，**echo** 加 \$ 能看某一条：

示范 1：读公告

```
$ echo $HOME
```

/home/<你的真实用户名> （示例输出，以你机器为准）

```
$ echo $PATH
```

/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin

\$ 的作用是"取公告内容"。PATH 是一串用冒号分隔的目录——你敲 ls 时，系统按这个顺序逐目录找叫 ls 的程序，找到就执行。

↑ 第 0 章的 "command not found" 谜底就在这：**你要的程序不在 PATH 的任何一个目录里**。装的东西找不到了？三种可能：没装、装了但 PATH 没有它的目录、名字记错了。

11.2 export：贴新公告

示范 2：把自定义目录加进 PATH

```
$ mkdir ~/mybin
```

```
$ export PATH="$HOME/mybin:$PATH"
```

```
$ echo $PATH
```

/home/<你的真实用户名>/mybin:/usr/local/bin:/usr/bin:...

注意写法：**新目录在前，\$PATH 原样带上**——如果把 \$PATH 丢了，等于把原公告全部覆盖，系统命令全都找不到（坑！）。

易错点：export 只对**当前终端**生效。关掉重开，公告没了。想让公告**永久有效**，把这行写进 ~/.bashrc（每次开终端自动执行的配置文件）。

11.3 alias 与 source

示范 3：起外号 + 立即生效

```
$ alias lh='ls -lh'
$ lh
```

```
total 12
-rw-r--r-- 1 student student 100M Aug 16 11:00 big.img （示例输出，实际以本机为准）
```

```
$ echo "alias lh='ls -lh'" >> ~/.bashrc
$ source ~/.bashrc
```

alias = 给长命令起外号。写进 `.bashrc` = 永久保存；**source** = 让当前终端立刻重读配置文件（不用重开终端）。注意：`ll`、`la` 这些名字 Ubuntu 已经预定义了别名，自定义外号避开它们。

🔒 安全闭环：①改前备份 `cp ~/.bashrc ~/.bashrc.backup`；②自己加的行用唯一注释标记（如 `# my-alias`），重复执行不会重复追加；③不想要了：删掉那两行再 `source`；④改乱想回退：`cp ~/.bashrc.backup ~/.bashrc` 再 `source`。验证恢复成功：`alias` 里 `lh` 消失。

本章对比：export 临时 vs 写进 ~/.bashrc 永久

做法	特点
export 变量=值	只对当前终端生效，关了就没——试东西用
写进 ~/.bashrc 再 source	永久生效，每次开终端自动加载——定下来后用

✍ 章末作业

1. `env` 看全部环境变量，找出 `HOME`、`USER`、`SHELL` 三条的值。
2. 建 `~/mybin`，把它的路径加进 `PATH`（别丢 `$PATH`），验证。
3. 自定义一个不撞名的别名（如 `lh`），写进 `.bashrc`，`source` 后验证。
4. 重开一个终端，确认别名还在（写进 `.bashrc` 的才能“永久”）。

🎯 章末自测

1. 敲命令报 `command not found`，有哪三种可能原因？
2. 判断：`export` 设置的环境变量在下次开机后仍然有效。（对/错）

✍ 对应练习册：进阶篇场景 1（`labservice.conf` 的 `LOGDIR` 就是环境变量的“文件版”用法）。

第 12 章 · 归档与压缩

本章目标：① 理解"归档"和"压缩"是两件事；② 会用 tar 打包（含 tar.gz）；③ 会 gzip / zip；④ 会用 diff 验证解包无误。

本章新命令：tar gzip gunzip zip unzip

12.1 两个概念先分开

概念	比喻	效果
归档	把一堆文件 装箱	多个 → 一个（大小几乎不变）
压缩	把箱子 抽真空	体积变小

tar 负责装箱，gzip 负责抽真空。**tar.gz = 先装箱再抽真空**（两步合成一条命令）。

示范 1：打包 + 压缩（背口诀）

```
$ mkdir -p ~/lab/ch2 && echo x > ~/lab/ch2/note.txt # 材料自足：没有就现造（跳章也能跑）
$ tar -czvf ~/lab/bak.tar.gz ~/lab/ch2
```

```
/home/<你的真实用户名>/lab/ch2/
/home/<你的真实用户名>/lab/ch2/note.txt
（tar 打印的是真实绝对路径，以你机器为准）
```

c=create 建包、z=gzip 压缩、v=verbose 显示过程、f=file 后面接文件名。口诀：**打包 czvf，解包 xzvf**（解包把 c 换成 x）。f 永远放最后，紧接文件名。

示范 2：解包 + 验证

```
$ rm -rf ~/lab/restore # 重复运行前先清旧解包目录（只清练习目录，确认无其他东西）
$ mkdir -p ~/lab/restore
$ tar -xzvf ~/lab/bak.tar.gz -C ~/lab/restore
$ RESTORED_DIR="$(find ~/lab/restore -type d -path '*/lab/ch2' -print -quit)" # 只取第一个匹配（-quit），不依赖通配展开，不误匹配多个用户目录
$ [ -n "$RESTORED_DIR" ] && diff -r ~/lab/ch2 "$RESTORED_DIR" && echo 还原一致
$ # 若没输出“还原一致”，先 echo "$RESTORED_DIR" 看取到的路径对不对
```

```
还原一致
```

解包后用 **diff -r 对比原目录**——无输出 = 一个不多一个不少。备份的意义在于“能原样还原”，验证这一步不能省（练习册 5-7 就是这道题）。

示范 3：gzip 单文件 / zip 跨平台

```
$ echo '这是一段用于压缩练习的文本，可以多写几行' > ~/lab/big.txt # 先自造要压缩的文件
$ gzip ~/lab/big.txt # → big.txt.gz
$ gunzip ~/lab/big.txt.gz # 解回来
$ zip -r ~/lab/for-windows.zip ~/lab/ch2
$ unzip ~/lab/for-windows.zip
```

gzip 只压单个文件（压完原文件消失，变成 .gz）；zip 是 Windows 也认的格式，跨系统交换文件用 zip。

易错点：c 和 x 写反 = 灾难（覆盖）；-f 必须最后；tar 本身只打包不压缩（tar.gz 里的 z 才负责压缩）。

章末作业

1. 把 ~/lab/ch2 打包成 tar.gz，观察大小，再解包到新目录。
2. 用 diff -r 验证解包目录和原目录一致。
3. 用 gzip 压一个文件，ls 看后缀变化，再用 gunzip 解回来。
4. 用 zip 打包发给"Windows 朋友"（虚拟），unzip 解包验证。

章末自测

1. 默写口诀：打包用什么，解包用什么？z 和 f 分别是什么？
2. 判断：tar 命令会自动压缩文件。（对/错）

 对应练习册：基础篇 5-6、5-7；进阶篇场景 2（归档大文件）。

第 13 章 • Shell 脚本基础

本章目标：① 理解"脚本 = 把敲过的命令装进文件"；② 会写带变量、if、for 的脚本；③ 会用 crontab 定时执行；④ 知道脚本的标准写法。

本章新命令：crontab (脚本语法：shebang、变量、if、for、函数、exit)

13.1 脚本是什么

你在终端一条条敲命令很累，而且容易敲错。把要敲的命令按顺序写进一个文件，以后执行这个文件就行——**这个文件就是 Shell 脚本**。它是运维自动化的起点：巡检、备份、告警，全是脚本。

示范 1：第一个完整脚本（资源巡检）

```
$ cat > ~/lab/check.sh << 'EOF'
```

```
#!/bin/bash
# 资源巡检脚本：磁盘、内存、负载
echo "==== 巡检开始 $(date) ====="
echo "--- 磁盘 ---"
df -h | grep -v "^tmpfs"
echo "--- 内存 ---"
free -h
echo "--- 负载 ---"
uptime
```

```
EOF
```

```
$ chmod +x ~/lab/check.sh
```

```
$ bash ~/lab/check.sh
```

逐行解读：**#!/bin/bash** (shebang，告诉系统用 bash 解释这个文件，永远第一行)；**#** 开头=注释；echo 打印提示；df/free/uptime 是你学过的命令（见第 7 章）原样放进来。**chmod +x** 给执行权限后才能 ./ 运行。

13.2 变量、if、for

示范 2: 变量 + 判断 + 循环

```
$ cat > ~/lab/check_disk.sh << 'EOF'

#!/bin/bash
# 磁盘告警：使用率超 80% 打印警告
THRESHOLD=80
USAGE=$(df / | tail -1 | awk '{print $5}' | tr -d '%')
if [ "$USAGE" -gt "$THRESHOLD" ]; then
    echo "警告：根分区已用 $USAGE%，超过 $THRESHOLD%！"
else
    echo "正常：根分区已用 $USAGE%"
fi
for user in alice bob; do
    echo "检查用户：$user"
done

EOF
$ chmod +x ~/lab/check_disk.sh && ./check_disk.sh
```

```
正常：根分区已用 26%
检查用户：alice
检查用户：bob
```

三个语法点：**变量**（THRESHOLD=80，**等号两边绝不能有空格**；用的时候加 \$）；**\$()** = 把命令的输出装进变量；**tr -d** '%' = tr 字符转换、-d 删除，把 % 号删掉（df 输出是 "26%" 带百分号，删掉才是纯数字，才能和 80 比大小）；**if/then/else/fi** 判断（注意 [] 里和条件之间有空格）；**for 变量 in 列表; do...done** 循环。

补两个脚本零件：

- ① **exit** = 结束脚本，并带一个“退出码”：exit 0（正常）、exit 1（出错）。别的程序（或你自己）可以读这个码判断脚本成没成功。
- ② **函数** = 把一段脚本起个名字，需要时叫它：check_disk() { df -h; }（定义），check_disk（调用）。函数让长脚本分块、可复用——进阶篇的巡检脚本就是“三个函数 + 主流程”的搭法。

函数与退出码：Shell 函数用“函数名(){ 命令;}”把命令封装起来；exit 0 表示正常结束，非 0 通常表示失败；\$? 可查看上一个命令的退出码。

13.3 crontab: 定时闹钟

crontab 让脚本定时自动跑。crontab -e 编辑（首次会问选编辑器）、crontab -l 查看。

示范 3：每天凌晨 2 点自动跑脚本

```
$ crontab -e
```

```
0 2 * * * /home/<你的真实用户名>/lab/check.sh >> /home/<你的真实用户名>/lab/check.log 2>&1
```

格式：**分 时 日 月 周 + 命令**（五个时间位，* = 任意）。这行=每天 2 点 0 分执行。**check.sh 就是本章 13.1 写好的巡检脚本——cron 只管按时叫你，跑什么还是你自己的脚本。**

✅ 安全闭环（照做）：①改前先备份 crontab -l > ~/lab/cron.backup；②实验行加唯一标记（如把 check.sh 改成每分钟跑一次：* * * * * /home/学生/lab/check.sh >> ...）；③等 1~2 分钟验证日志新增后，立即删掉该实验行；④核对只剩原有任务：crontab -l；⑤若改乱，恢复：crontab ~/lab/cron.backup。cron 无“运行错误提示”，验证只能看脚本自己写的日志或邮件。

易错（重要）：cron 的环境和你的终端**不是同一个**——它对 ~ 的展开不可靠。定时任务里的路径**一律写绝对路径**（/home/<你的真实用户名>/...，不写 ~/...）。这是练习册进阶篇 5-4 的考点，也是实战第一大坑。

本章对比：脚本 vs 手动敲

对象	特点 / 适用场景
手动一条条敲	一次性操作；边看边敲边改
写成脚本 (.sh + chmod +x)	可重复、可定时 (crontab)、可交给别人跑——一切自动化从写脚本开始

13.4 易错点

坑 1：变量赋值 NAME=student 等号两边有空格 → 报错 command not found。

坑 2：写好的脚本忘 chmod +x → Permission denied。

坑 3：脚本第一行 #!/bin/bash 忘了写，在部分环境会出怪问题。

✍ 章末作业

1. 写一个脚本：打印"今天日期 + 磁盘使用率 + 内存剩余"，跑通。
2. 给脚本加变量和 if：内存可用低于 20% 时打印警告。
3. 用 for 循环遍历 3 个文件，对每个打印"文件 X 的大小是..."。
4. 🌐 crontab -e 加一条每分钟执行的测试任务（用绝对路径！），观察日志后删掉它。

🎯 章末自测

1. 脚本第一行 #!/bin/bash 是什么？crontab 里为什么必须用绝对路径？
2. 判断：脚本写好后不需要执行权限也能 ./ 运行。（对/错）

✍ 对应练习册：进阶篇场景 5（两个脚本 + cron 收尾，直通"5 个 Shell 脚本"任务）。

附录 A · 章末作业与自测答案

先自己做，再对答案。自测题给答案；动手作业给“检查点”（做到什么程度算对）。

第 0 章

自测 1： \$ 表示当前是普通用户；# 表示当前是 root（管理员）身份。

自测 2： 错。ls 是命令（动词），-l 是选项（怎么做），/home 是参数（对谁做）。

✓ 作业检查点：① pwd 输出 = /home/<你的真实用户名>，whoami = 你的真实用户名（两者对应）；② 能说出 3 个选项及作用；③ history 能看到今天敲过的命令，↑ 能调出上一条并再次执行；④ 报错原文以 command not found 结尾。

第 1 章

自测 1： 绝对路径从树根 / 出发写全称，走到哪都不变；相对路径从当前目录出发，站在不同位置同一个写法指向不同地方。

自测 2： 错。cd .. 是去“上一级目录”；cd - 才是回“上一个待过的目录”。

✓ 作业检查点：① cd /etc 后 pwd 输出 /etc，cd ~ 后 pwd 输出 /home/<你的真实用户名>；② 两种路径进入同一目录后 pwd 输出相同；③ 找出 3 个 . 开头文件（.bashrc、.profile 等）；④ / 下能看到 home、etc、var、usr、bin 等目录。

第 2 章

自测 1： 文件系统里“改名”就是把它在目录树里的记录从旧名字移到新名字——和 mv 的实现完全一致，所以 Linux 不需要单独 rename 命令。

自测 2： 错。rm 删除不可恢复，没有回收站；恢复软件成功率极低。

✓ 作业检查点：① 一条命令建成 x/y/z（mkdir -p）；② 用 ls 对比两目录清单一致（diff -r 留到学完第 3 章 3.4 后补做）；③ ls 里 second-note.txt 存在、note2.txt 消失；④ rmdir 报“Directory not empty”，rm -r 后目录消失。

第 3 章

自测 1： C（tail -f 实时跟踪）。cat 会刷屏，less 要手动翻，tail -f 直接等新报错。

自测 2： 错。说反了：grep 按内容搜，find 按文件名/属性找。

✓ 作业检查点：① less 里空格/b/q/搜索都能操作；② grep -ci error 和 grep -i 数出的行数一致；③ find ~ -name "*.txt" 列出家目录所有 txt；④ 改动前 diff 无输出，改动后 diff 显示差异行。

第 4 章

自测 1： 管道 | 连接命令与命令（数据不落地）；重定向 > 连接命令与文件（数据写进文件）。

自测 2： 错。uniq 只对相邻重复行生效，乱序文件必须先 sort。

✓ 作业检查点：① cat 验证 > 覆盖、>> 追加（和示范 2 现象一致）；② uniq -c 每行 = 数字 + 词；③ awk '{print \$1}' 输出全是第一列；④ 管道链改造后输出 3 个词，用 head -3 收尾。

第 5 章

自测 1： 其他人权限是 r--（只读）；数字法是 4，整串是 754。

自测 2： 错。说反了：sudo 是借 root 权限执行一条命令，su 是切换成另一个用户。

✓ 作业检查点：① groups alice 和 groups bob 各有输出；② groups alice 结果里含 sudo；③ ls -l 两次都显示 -rwxr-xr-x（755）；④ alice 读 bob 的文件被拒绝或按权限位受限（前提：alice 不在该文件属组内）。

第 6 章

自测 1： kill 发 SIGTERM（程序可清理后优雅退出）；kill -9 发 SIGKILL（内核直接处决，无清理机会）。对数据库等直接 -9 可能损坏数据，所以 -9 是最后手段。

自测 2： 错。nohup 是“挂断不死”（关终端进程继续跑）；忽略错误输出是 2>&1 和 > 的事。

✓ 作业检查点：① 找到 bash 进程并读出 PID、%CPU；② sleep 300 已被 kill（ps 再查已消失）；③ run.log 有清理记录，kill -9 那次无新增记录；④ 重开终端 ps 还能看到 nohup 任务。

第 7 章

自测 1: df 看整个分区（整体），du 看具体目录（明细）。排查先 df 定位满的分区，再 du 逐层下钻。

自测 2: 错。Linux 没有盘符，新硬盘要挂载（mount）到某个目录。

☑ 作业检查点：① df -h 里根分区 Use% 有数字；② 先 ls -la ~ 再 du -sh ~/* 排出前 3 名（注意 . 开头的隐藏项也要看）；③ ls -lh 显示 big.img 为 100M；④ lsblk 的 sda1 与 df 的 /dev/sda1 对应得上。

第 8 章

自测 1: ① ping 目标 IP；② nc -zv 目标 端口 / ss -tlnp；③ ps aux | grep / fuser -v 端口；④ tail -f 对应日志。

自测 2: 错。对方可能禁 ping（防火墙丢 ICMP），服务却正常；应继续用 nc 测端口。

☑ 作业检查点：① ip addr 里 inet 后是本机 IP，ip route 里 default via 后是网关；② ss -tlnp 里认出 22 端口 sshd；③ nc -zv 22 端口 succeeded、9999 端口 refused；④ curl -I 能看到 HTTP/1.1 200 等状态行。

第 9 章

自测 1: update 刷新软件列表（不装不升级）；upgrade 把已装软件升级到新版本。

自测 2: 错。锁报错=另一个 apt 在跑或上次异常中断；磁盘满会报 No space left on device。

☑ 作业检查点：① 装的软件能直接运行；② apt list --installed | grep htop 有结果；③ remove 后 list 里无 htop；④ 终端 2 出现 Could not get lock 报错。

第 10 章

自测 1: start=现在启动一次；enable=设开机自启。想重启机器后自动运行：systemctl enable 服务。

自测 2: A。先 status 一眼定位死活，再 journalctl 找原因；C、D 都信息不全。

☑ 作业检查点：① Loaded/Active/Main PID 三行能逐行解释；② 每次操作后 status 的 Active 字段与动作对应（stop 后 inactive，start 后 active）；③ -p err 只显示 error 级别；④ status 输出里含 enabled 字样 = 开机自启。

第 11 章

自测 1: ① 没装这个软件；② 装了但它的目录不在 PATH 里；③ 命令名拼错了。

自测 2: 错。export 只对当前终端生效；想永久生效要写进 ~/.bashrc。

☑ 作业检查点：① env 里找到 HOME/USER/SHELL 三条；② echo \$PATH 以 /home/<你的真实用户名>/mybin 开头，且原目录都在（没丢 \$PATH）；③ lh 能列出详细文件；④ 新开终端 lh 依然可用。

第 12 章

自测 1: 打包 czvf、解包 xzvf；z=用 gzip 压缩，f=指定文件名（放最后）。

自测 2: 错。tar 只归档不压缩；tar.gz 里的 z 才压缩。

☑ 作业检查点：① ls 里出现 bak.tar.gz；② diff -r 无输出（一个不多一个不少）；③ 文件后缀变 .gz，gunzip 后还原；④ unzip 能解出全部文件。

第 13 章

自测 1: #!/bin/bash 是 shebang，告诉系统用 bash 解释脚本。cron 环境对 ~ 展开不可靠，路径必须写绝对路径才能保证找得到。

自测 2: 错。必须有执行权限（chmod +x）才能 ./ 运行；或改用 bash 脚本名 方式运行。

☑ 作业检查点：① 脚本输出含日期、磁盘、内存三项；② 临时把阈值改大（如 90）能触发警告分支，再改回 20；③ 三个文件各行打印出“文件 X 的大小是...”；④ crontab -l 能看到任务，日志每分钟新增，删除后 crontab -l 无该项。

附录 B · 术语表

术语	解释
终端 Terminal	和 Linux 对话的窗口
提示符	student@ubuntu:~\$ 这串符号，出现=等你敲命令
命令/选项/参数	做什么 / 怎么做 / 对谁做
发行版	Ubuntu、CentOS 等（同内核，不同包装）
API	程序之间"说话的接口"：你按约定发请求，对方按约定回数据（curl、requests 调的都是 API）
目录树	一切文件的组织结构，根是 /
一切皆文件	连设备、磁盘都是文件，统一操作
绝对/相对路径	从 / 出发 / 从当前位置出发
家目录 ~	你的地盘 /home/<你的真实用户名>
通配符	*任意一串、? 任意一个
管道	把前一个命令的输出喂给后一个
重定向 > / >>	输出到文件（覆盖 / 追加）
权限位 rwx	读 / 写 / 执行；分属主、属组、其他人三组
root / sudo	管理员 / 借管理员权限执行一条命令
进程 / PID	运行中的程序 / 它的工号
SIGTERM / SIGKILL	请退出（可清理） / 当场击毙（无机会）
前台 / 后台	占着终端 / 不占终端（&）
nohup	关终端也不死的后台运行
挂载 mount	把磁盘/分区接到目录树上
IP / 端口	门牌号 / 窗口号
监听 LISTEN	服务开着窗口等人连
包管理器 apt	应用商店+自动处理依赖
systemd	开机大管家，管所有服务
服务 unit	常驻后台的程序 / 它的档案文件
开机自启 enable	重启机器后自动拉起
环境变量	全局公告栏（HOME、PATH）
PATH	找命令的目录清单，冒号分隔
.bashrc	每次开终端自动执行的配置
alias	命令外号
归档 / 压缩	装箱 / 抽真空
shebang	脚本第一行 #!/bin/bash
cron	定时闹钟（分时日月周）
JSON	数据交换格式，{键:值}（API 返回常用）
四层排查法	主机→端口→进程→日志
Shell（壳）	包在内核外的"命令翻译官"，负责把你敲的命令翻译给内核执行。见第 0 章。
内核（kernel）	Linux 最核心的程序，管理 CPU、内存、硬盘等硬件。见第 0 章。

附录 C · 命令速查总表 (按章)

章节	命令
第 0 章	pwd ls echo --help man clear history exit whoami
第 1 章	pwd ls cd (~ .. -)
第 2 章	touch mkdir(-p) cp(-r) mv rm(-r -f) rmdir
第 3 章	cat less head tail(-f) wc file grep(-i -r -n -v -c) find(-name -size) diff(-r)
第 4 章	> >> 2>&1 sort uniq(-c) awk('{print \$1}')
第 5 章	whoami id su sudo useradd passwd usermod(-aG) groupadd groups userdel last chmod(符号/数字) chown chgrp
第 6 章	ps aux top htop pgrep kill killall jobs bg fg nohup fuser(-v)
第 7 章	df(-h) du(-sh) lsblk mount fallocation dd mkfs
第 8 章	ip addr ip route ping(-c) ss(-tlnp) curl(-I -o) wget nc(-zv) traceroute nslookup ssh scp
第 9 章	apt update/install/remove/search/upgrade/list dpkg
第 10 章	systemctl(status start stop restart reload enable disable) journalctl(-u -f -p)
第 11 章	env echo(\$变量) export source alias (~/.bashrc)
第 12 章	tar(-czvf -xzvf) gzip gunzip zip unzip
第 13 章	crontab(-e -l) chmod +x (脚本: #!/bin/bash 变量 if for \$())

附录 D · 章 ↔ 练习册对应表（含新命令对齐）

本书章节	练习册	对齐的新命令
第 0~2 章	基础篇 阶段 1 (1-1~1-8)	—
第 3~4 章	基础篇 阶段 2 (2-1~2-8)	grep、wc、管道、重定向
第 5 章	基础篇 阶段 3 (3-1~3-8)	chmod 数字算法、chown、usermod
第 6~7、9、10 章	基础篇 阶段 4 (4-1~4-8)	fuser、htop、df/du
第 3~4、12 章	基础篇 阶段 5 (5-1~5-8)	awk、find、sort、uniq、tar
第 3、8、10 章	进阶篇 场景 1	grep、ss、journalctl
第 7、12 章	进阶篇 场景 2	fallocate、dd、tar、df/du
第 8 章	进阶篇 场景 3	ip route、nc、ss、ping
第 6、10 章	进阶篇 场景 4	fuser、nohup、systemctl
第 13 章	进阶篇 场景 5	crontab（绝对路径）、脚本语法

《Linux 入门》教科书版 · 与《Linux 指令练习册》基础篇 / 进阶篇、《英语单词手册》《日志报错解读手册》《Python 内置函数手册》配套使用 · v2.4 (2026-09)